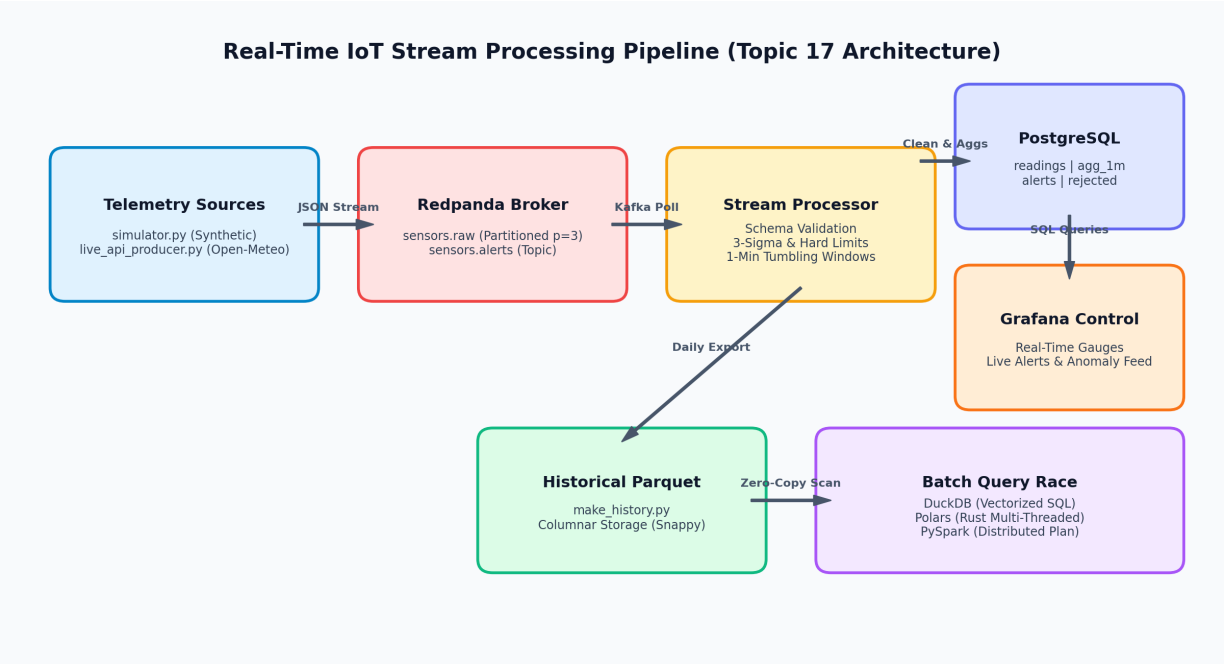
PRACTICAL CASE STUDY REPORT

Real-Time IoT Stream Processing

A hands-on pipeline with Redpanda, Python, PostgreSQL, Grafana, Kubernetes, and batch analytics

Submission review: implementation evidence and honest checklist status



Prepared from the repository contents and recorded benchmark artifacts. Date: 12 October 2026.

Executive Summary

This project implements a practical real-time IoT pipeline: telemetry is produced as JSON, transported by Redpanda, validated and analyzed by a Python consumer, stored in PostgreSQL, and displayed in Grafana. It also includes an Open-Meteo API producer, historical Parquet analytics, Docker Compose and Kubernetes manifests, and benchmark scripts.

The checklist is substantially covered, but the evidence does not support claiming every item is complete. The Spark job is configured for local mode rather than a multi-node cluster; the reported streaming throughput and recovery results are generated through mock code paths; and no 8-12 slide presentation is present. This report labels those limitations explicitly.

Project at a glance

Area	What the repository provides
Streaming	Redpanda topics, Python simulator and processor, validation, anomaly alerts, one-minute aggregates, PostgreSQL tables, and Grafana provisioning.
Data sources	Synthetic sensor events, generated historical Parquet, and an optional live weather API producer.
Analytics	DuckDB and Polars query comparison; PySpark example with transformations, actions, repartitioning, and caching.
Deployment	Docker Compose for local services and Kubernetes manifests for the streaming stack.
Evidence caveat	Some benchmark/failure artifacts are mock simulations and should not be presented as production-broker measurements.

1. Introduction

Buildings and connected facilities produce continuous readings such as temperature and power demand. Operators need a pipeline that can accept events, reject malformed records, identify unusual readings, preserve useful data, and expose current conditions for inspection. The project models this end-to-end workflow and adds historical query comparisons.

2. Problem Statement

The problem is to collect time-stamped readings from multiple devices, validate their structure and values, detect threshold or statistical anomalies, aggregate recent readings, retain clean and rejected records separately, and make both live monitoring and historical analysis available. Operationally, the system should be repeatable on a developer machine or local Kubernetes cluster.

3. Methodology

The review followed the actual source, deployment manifests, and result artifacts in the repository. The pipeline is separated into ingestion, broker transport, stream processing, persistence, dashboarding, and historical analytics. Reported measurements are distinguished from mock-path demonstrations.

Evaluation questions

- Can the services be deployed and connected through the documented local or Kubernetes setup?
- Do transformations include validation, filtering, anomaly logic, and grouped/windowed results?
- Are cache, partition, or shuffle changes present, and are their performance effects measured?
- Do the charts and tables represent live broker measurements or a local simulation?

- Does the submission contain the requested report, code, screenshots, and presentation?

4. System Architecture

Figure 1 summarizes the intended data flow. The PNG is the repository architecture diagram and is an explanatory diagram, not a captured runtime topology.

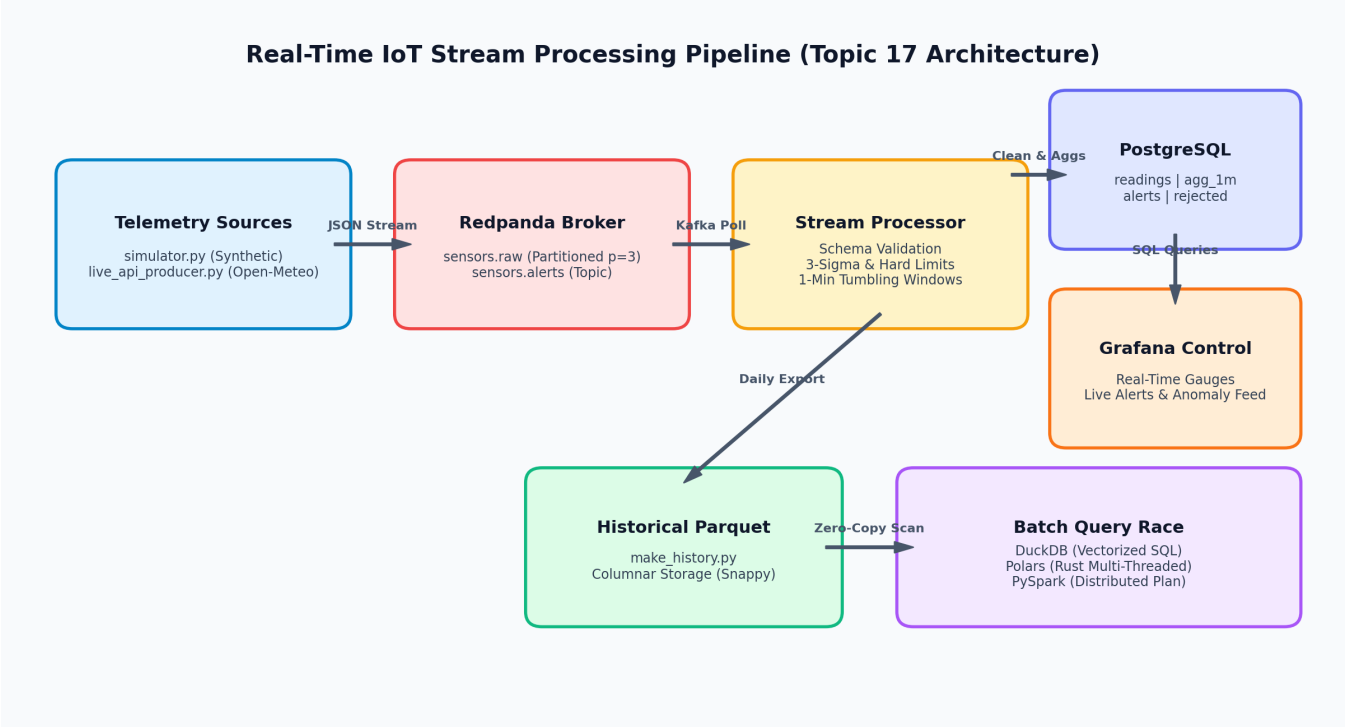


Figure 1. Pipeline architecture diagram included in the project documentation.

5. Implementation and Explanation

5.1 Environment and deployment

The project uses Python for the producer and consumer, Redpanda for Kafka-compatible messaging, PostgreSQL for queryable recent data, Grafana for dashboards, and Kubernetes YAML manifests for container orchestration. The repository also contains Docker Compose configuration. The Spark example requires PySpark plus a Java runtime for actual Spark execution. No Hadoop cluster configuration or multi-node Spark deployment is evidenced.

Kubernetes manifests cover the namespace, Redpanda, PostgreSQL, stream processor, simulator, Redpanda Console, and Grafana. This is a meaningful Kubernetes deployment of the streaming services. Spark analytics are not included as a Kubernetes workload in the manifests.

5.2 Data loading and event contract

Streaming events use JSON fields for device identifier, room, event timestamp, temperature, and power. The simulator can generate ordinary events and inject spikes or malformed rows. The optional live producer polls Open-Meteo for current weather at five city coordinates and maps temperature and wind into sensor-like fields; power is a calculated proxy. This is real API input, but it is not a measurement from the building sensors modeled by the project. Historical benchmark data is generated as Parquet. A dedicated CSV ingestion path was not identified.

5.3 Distributed processing, transformations, and actions

The stream processor validates incoming messages, routes bad rows to a rejected table, detects threshold and rolling-history anomalies, emits alerts, and writes clean readings and one-minute aggregates. The analytics scripts use grouped queries in DuckDB and Polars. The Spark script reads Parquet and demonstrates filter, groupBy, withColumn, repartition, persist/cache, count, collect, and show operations.

Important scope detail: Spark is explicitly started with master local[*], which uses local machine cores. It is not evidence of a multi-node distributed Spark run. If Spark initialization fails because Java is unavailable, the code falls back to a Spark-emulation path. Therefore describe the Spark portion as a local-mode implementation/demo unless a cluster execution is separately captured.

5.4 Performance optimizations

The Spark code repartitions by device_id into four partitions, configures four SQL shuffle partitions, and persists a partitioned DataFrame in MEMORY_ONLY storage before timing a repeated filter/count action. These are concrete optimization mechanisms. Their effects should be supported by an actual Spark run with recorded uncached and cached timings; the repository performance table does not include those values. No cluster-mode comparison is included.

5.5 Results capture

The repository includes architecture and dashboard-preview images plus latency, throughput, and engine-comparison charts. The dashboard image is generated as a mock preview, not a screenshot captured from a running Grafana instance. A user-provided Grafana view showed dashboard panels, but most time-series panels were empty at that moment. Capture a fresh live dashboard screenshot after data has populated those panels for stronger submission evidence.

6. Results and Analysis

6.1 DuckDB versus Polars historical queries

The saved report records a four-query comparison on a 100,000-row generated Parquet dataset, with median timing across three runs in the benchmark script. Polars is faster on all four recorded queries in this run. These are single-machine engine timings, not distributed-cluster performance results.

Query	DuckDB (s)	Polars (s)	Recorded ratio
Hourly room temperature average and max	0.0354	0.0098	3.61x
Daily room power sum	0.0236	0.0094	2.49x
Top devices by anomaly count	0.0073	0.0041	1.76x
Reading count by device	0.0062	0.0041	1.52x

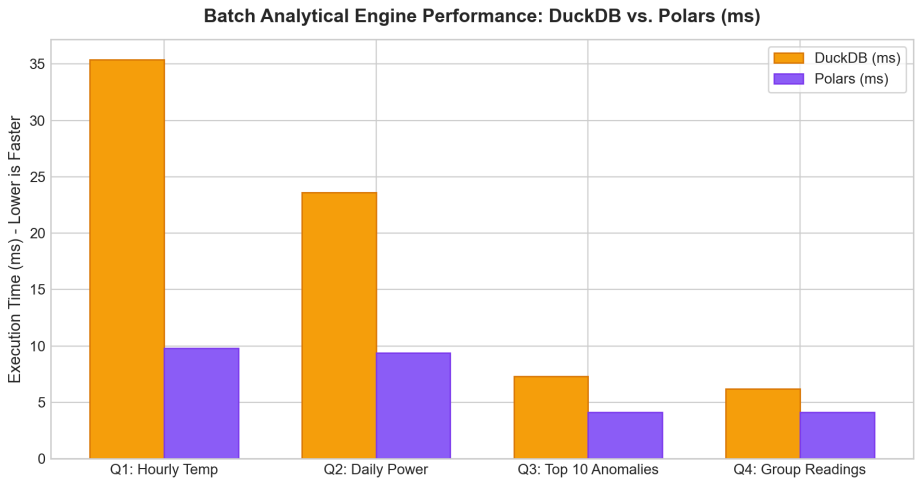


Figure 2. Saved local batch-engine comparison chart.

6.2 Streaming benchmark values and their limits

The result markdown records 3,000 events for each simulated device scale. The capture script invokes the simulator in mock_mode=True, so the values below measure a local mock publishing path rather than confirmed Redpanda broker acknowledgments or end-to-end pipeline processing. Treat these figures as code-path timing only.

Devices	Events	Reported msg/s	p50 ms	p95 ms	p99 ms
10	3,000	10,298.3	0.0508	0.2157	0.6835
50	3,000	11,828.16	0.0332	0.2395	0.8096
100	3,000	9,194.83	0.0312	0.2909	1.2926

The reported rate is not monotonic: the 50-device case is highest and the 100-device case is lowest, while p99 latency rises with scale. Because the measured path is mocked, these figures do not establish broker capacity or deployment scaling.

6.3 Fault recovery evidence

A saved result reports 100 events processed as 40 before and 60 after a simulated crash, with zero missing or duplicated events. Source inspection shows failure_test.py uses an in-memory list and manually advances a variable named committed_offset; it does not start Redpanda or kill/restart a Kafka consumer. This is a logic demonstration only and cannot substantiate real broker offset recovery or zero-loss delivery.

6.4 Practical interpretation

- The streaming architecture and Kubernetes service manifests demonstrate a broad, integrated implementation.
- The DuckDB/Polars table gives a useful local query comparison, but it is limited to one generated dataset and one machine.
- The benchmark and failure claims need real broker-backed reruns before they can be stated as measured streaming performance or recovery guarantees.
- Spark optimization mechanisms exist in code; comparative cache/partition measurements and cluster execution remain to be captured.

7. Checklist Assessment

Status reflects repository evidence reviewed for this report. “Partial” means implementation exists but the requested execution proof or full scope is missing.

Checklist item	Status	Evidence / gap
Environment setup (Spark / Hadoop / Python)	Partial	Python and streaming services are present; Spark local-mode code exists. No Hadoop setup or multi-node Spark cluster evidence.
Load dataset (CSV / JSON / API)	Met with scope note	JSON events and Open-Meteo API producer exist; historical data is generated as Parquet. No dedicated CSV loader found.
Distributed processing	Partial	Redpanda streaming services run in Kubernetes; Spark is local[*], not a cluster execution.
Transformations and actions	Met	Stream validation/aggregation and Spark transformations/actions are implemented in code.
Performance optimization	Partial	Repartition/cache/shuffle settings exist; no saved Spark cache comparison or cluster-mode comparison.
Capture results (screenshots/output)	Partial	Charts and a dashboard preview exist; the Grafana preview is a mock image and live chart evidence should be recaptured.
Analyze performance	Partial	DuckDB/Polars timings are recorded; stream throughput and recovery figures use mock code paths.
Report (PDF)	Met	This report supplies the requested PDF sections.
Code and output screenshots	Partial	Code and generated charts exist; ensure annotated scripts and authentic runtime screenshots are included in final submission.
Presentation (8-12 slides)	Not evidenced	No presentation deck was found in the repository reviewed.
Bonus: real-time data/API	Met with scope note	Open-Meteo API producer exists; its power field is a proxy and the data represents weather locations.
Bonus: Kubernetes deployment	Met	Kubernetes manifests cover the core streaming stack and UI services.
Bonus: compare cluster modes	Not met	No Spark standalone/YARN/Kubernetes mode comparison was found.

8. Conclusion

The project demonstrates a substantial end-to-end IoT streaming implementation and covers many practical requirements, including event ingestion, processing, storage, dashboarding, historical analytics, and Kubernetes deployment. The answer to “did we fulfill all of this?” is: not completely, based on the artifacts currently in the repository. The most important remaining submission work is to capture real Redpanda-backed throughput and consumer-recovery evidence, run and record Spark optimization measurements (and a cluster mode if required), capture a populated live Grafana dashboard, and create the 8-12 slide presentation.

Recommended final evidence to add

- Record the exact cluster/context and versions used; include actual pod readiness and topic output.
- Run a non-mock producer benchmark with broker delivery callbacks and report end-to-end consumer or database counts.
- Run an actual consumer restart against Redpanda and compare produced, unique stored, and duplicate event identifiers.
- Record Spark local cache/partition timings, then compare against a second cluster mode only if that environment is available.
- Capture Grafana with a useful time range after readings and aggregates have populated the panels.
- Create an 8-12 slide deck with the architecture, method, real results, limitations, and key findings.

9. Sources and Reproducibility Notes

This assessment is based on repository artifacts: README.md, k8s/README.md and Kubernetes manifests, processor.py, simulator.py, live_api_producer.py, spark_analytics.py, analytics.py, capture_results.py, failure_test.py, docs/results/RESULTS_AND_ANALYSIS.md, and images under docs/screenshots/. The Open-Meteo producer requests its public forecast endpoint at api.open-meteo.com. For reproducibility, preserve the scripts, configuration, dataset row count, software versions, run command, and raw output alongside future measurements.